

AI Usage Note

PS-05 — RCA Drafter | Infinite AI Prototype Challenge

Summary

AI assistants were used throughout the development of RCA Drafter — for architecture design, code generation, prompt engineering, and structured output formatting. This note documents what worked, what failed, and the key prompts used.

What AI Helped With

1. Architecture & Design

What AI did: Generated the initial multi-agent architecture with specialized agents for timeline, log, and code analysis, plus a coordinator and synthesizer.

Outcome: Clean separation of concerns with MCP-inspired tools. Reduced ambiguity about task delegation and coordination, enabling parallel execution.

2. Code Generation

What AI did: Generated utility functions for regex-based timeline parsing, log extraction, and git diff parsing, plus all agent classes with specialized LLM prompts.

Outcome: Saved 2–3 hours of manual regex writing and edge-case handling. Multiple agent classes generated with clear responsibilities.

3. Structured Output Generation

What AI did: Created the synthesis agent logic that combines findings from all agents into the final RCA Markdown.

Outcome: Consistent, professionally formatted reports following SRE best practices with all required sections grounded in evidence.

4. Flask Web App

What AI did: Built the web form interface and API endpoints for incident data submission.

Outcome: Non-technical users can generate RCAs without command-line knowledge.

What AI Got Wrong — & How We Fixed It

Initial LLM Prompt Too Generic

Problem	First prompt asked for free-form RCA text — output was narrative-style, not structured.
Fix	Rewrote prompt to require JSON output with specific keys.
Lesson	Always constrain LLM output format to what you need programmatically.

Missing Error Handling in MCP Tools

Problem	Parse functions crashed on malformed input (missing timestamps, blank lines).
Fix	Added null checks, default values, and filters for short/empty lines.
Lesson	Test parsing tools with intentionally bad data, not just happy paths.

Ollama Connection Not Validated

Problem	App started but crashed when calling the LLM if Ollama was not running.
Fix	Added validate_ollama_connection() method with clear error messages.
Lesson	Always provide a health check for external dependencies.

LLM Hallucinated Numbers

Problem	Action items referenced non-existent log entries or fabricated statistics.
Fix	Added evidence-grounding: "Only quote from provided data, never fabricate."
Lesson	Use low temperature (0.3) for factual tasks; explicitly forbid fabrication.

Best Prompts Used

Prompt 1 — Multi-Step Agent Loop Orchestration

Status: ✓ Successful

```
You are an expert SRE analyzing a production incident. INCIDENT: {incident_title}
TIMELINE EVENTS: {json.dumps(timeline_events, indent=2)} ERROR LOGS EXTRACTED:
{json.dumps(log_errors, indent=2)} RECENT CODE CHANGES: {json.dumps(code_changes,
indent=2)} CROSS-REFERENCE INSIGHTS: {json.dumps(cross_ref, indent=2)} Provide analysis
covering: 1. PRIMARY ROOT CAUSE: single deepest technical cause (not a symptom) 2.
CONTRIBUTING FACTORS: 2-3 factors that made the problem worse 3. EVIDENCE: quote
specific log lines and code changes 4. SEVERITY: P1/P2/P3 based on impact 5. ACTION
ITEMS: 4-6 specific, actionable remediation steps Format as JSON: {primary_root_cause,
contributing_factors, evidence, severity, action_items, lessons_learned} Respond ONLY
with valid JSON, no additional text.
```

Why it worked: Structured extracted data provided upfront. JSON output constrained. Fabrication explicitly forbidden. Clear severity criteria.

Prompt 2 — Root Cause Identification (Revised)

Status: ✓ Successful after iteration

Review the PRIMARY TIMELINE EVENT where failure occurred. Cross-reference with ERROR LOGS to find: Was this caused by (A) recent code change, (B) config change, or (C) environmental factor? Quote the specific log line that confirms your hypothesis. Then identify: Is this a SYMPTOM or a ROOT CAUSE? Answer: {brief answer}

Why it worked: Explicit event correlation (timeline→logs→code). Forced distinction between symptom and root cause. Evidence quoting prevents hallucination.

Prompt 3 — Action Item Generation

Status: ✓ Successful

For each identified root cause and contributing factor, generate ONE action item. Format: Action: [specific fix or process improvement] Priority: [High|Med|Low] Owner: [team/role] Constraints: - Specific enough to assign to one person (NOT "improve monitoring") - Must trace directly to a root cause above - Doable within 1-2 weeks

Why it worked: Specificity requirement prevented generic items. Traceability kept focus on root causes. Effort scoping made items realistic.

Model Settings Used

Setting	Value	Reason
Temperature	0.3	Low — deterministic, factual analysis
Model	phi3:mini (Ollama)	Good balance of speed and quality
Timeout	60 seconds	Allows complex multi-step reasoning

Lessons for Future Use

Lesson	Applies To	Recommendation
Constrain LLM output format early	All text generation	Always request JSON, not free text
Validate external dependencies	Integration tasks	Add health checks for APIs and LLM services
Test with bad data	Data parsing	Create test cases with malformed/missing fields
Ground LLM in evidence	Factual tasks	Provide structured data; forbid fabrication

Use low temperature	Deterministic tasks	Set temp ≤ 0.3 for analysis tasks
Iterate on prompt clarity	Prompt engineering	Test multiple phrasings; measure output quality

Overall AI contribution: Reduced development time from ~4 days to ~16 hours (75% faster) while enabling a more scalable and maintainable architecture.